



Wednesday, 12th November 2025

Abstract

This experiment investigates the control of a DC motor using an L298P Motor Driver Shield interfaced with an Arduino Uno via GPIO pins. The objectives were to understand the operation of the L298P H-bridge driver, control motor direction through digital GPIO signals, and implement Pulse Width Modulation (PWM) to vary motor speed. The motor driver's ENA/ENB pins were used for speed control through PWM signals, while IN1/IN2 and IN3/IN4 determined directional control for Motor A and Motor B. The procedure involved assembling the shield onto the Arduino, connecting the DC motors, uploading Arduino code via the Arduino IDE, and modifying PWM values to study speed variations. The results showed that increasing the PWM duty cycle increased motor speed proportionally, while reversing the logic state of the direction pins successfully changed rotation direction. The experiment concludes that the L298P motor driver effectively enables independent control of motor speed and direction, demonstrating the importance of PWM in providing efficient and precise motor speed regulation in mechatronic systems.

Table Of Content

Abstract.....	2
Table Of Content.....	3
Introduction.....	4
Materials and Equipments.....	5
Experimental Setup.....	6
Methodology.....	7
Data Collection.....	9
Data Analysis.....	10
Results.....	11
Discussion.....	13
Conclusion.....	14
Recommendations.....	15
References.....	17
Appendices.....	18
Acknowledgments.....	20
Student Declaration.....	21

Introduction

OBJECTIVE

The objective of this experiment is to control the speed and direction of a DC motor using the L298P Motor Driver Shield and an Arduino Uno. Specifically, the experiment aims to utilize GPIO pins for direction control and PWM (Pulse Width Modulation) signals for speed regulation. By completing this task, students will learn how to interface a motor driver with a microcontroller, write control code using the Arduino IDE, and observe changes in motor behavior based on different input signals.

BACKGROUND INFORMATION AND RELEVANT THEORY

A DC motor converts electrical energy into rotational mechanical energy. To control its rotation direction and speed electronically, an H-bridge motor driver is commonly used. The L298P Motor Driver Shield is based on the L298P H-bridge IC, which allows control of two DC motors simultaneously. Direction control is achieved using GPIO pins (e.g., IN1 and IN2), where different HIGH/LOW combinations determine whether the motor spins forward, backward, or stops. Speed control is facilitated by PWM, a technique that rapidly switches the motor's power on and off. The percentage of time the signal remains ON during each cycle, known as the duty cycle, determines the average output voltage and thus the motor speed. A higher duty cycle results in a higher effective voltage and faster motor rotation.

HYPOTHESIS

It is hypothesized that increasing the PWM duty cycle will result in a proportional increase in the DC motor's rotational speed. Additionally, changing the logic state of the direction control pins (IN1 and IN2) will successfully reverse the motor's direction. It is also expected that setting both direction pins to the same logic level (e.g., HIGH–HIGH) will cause the motor to stop due to an electrical braking effect within the H-bridge.

Materials and Equipments

The following components and equipment were used to carry out the DC motor control experiment using the L298P Motor Driver Shield:

1. **Arduino Uno** – Microcontroller used to execute the control program.
2. **L298P Motor Driver Shield** – Motor driver module based on the L298P H-bridge IC, used to control motor direction and speed.
3. **DC Motor (6V–12V)** – One or two DC motors used as the mechanical load for testing.
4. **External Power Supply (9V or 12V)** – Provides sufficient current to drive the motors (motors are not powered directly from Arduino).
5. **Breadboard (if required)** – Used for additional wiring connections.
6. **Jumper Wires** – Used to connect GPIO pins and power lines.
7. **USB Cable (Type A to B)** – For uploading code and powering the Arduino from a computer.
8. **Computer with Arduino IDE Installed** – Used to write, compile, and upload Arduino code to the microcontroller.

9. *(Optional)* **Tachometer** – For measuring motor speed (RPM), or visual estimation may be used if unavailable.

Experimental Setup

For this experiment, the Arduino Uno microcontroller served as the central control unit. The L298P Motor Driver Shield was mounted directly on top of the Arduino Uno, aligning its pin headers with the Arduino's digital and power pins. This allowed direct electrical connection between the Arduino and the motor driver without requiring additional jumper wires for signal pins. A DC motor was connected to the Motor A terminal block on the L298P shield. For optional dual-motor testing, a second motor could be connected to the Motor B terminal block.

An external DC power supply (9V or 12V) was connected to the screw terminal input on the motor driver shield to provide sufficient current for driving the motor. The ground (GND) of the external power supply was shared with the Arduino's ground to ensure a common reference.

GPIO pins IN1 and IN2 on the driver controlled the motor direction through digital logic signals, while the ENA pin was connected to a PWM-capable pin (D3) on the Arduino to control motor speed. The Arduino was connected to a computer via a USB cable, which enabled programming and serial monitoring through the Arduino IDE.

(Insert Circuit Diagram Here)

Pin Configuration Summary:

Function	L298P Shield Pin	Arduino Pin
Motor A Speed (PWM)	ENA	D3 (PWM)
Motor A Direction (1)	IN1	D12
Motor A Direction (2)	IN2	D13
External Power Input	—	+9V / +12V
Common Ground	—	GND

Once all connections were confirmed, the Arduino code was uploaded to the microcontroller.

The motor was observed rotating in forward and reverse directions depending on the logic signals, and its speed was adjusted by modifying the PWM duty cycle within the program.

Methodology

This experiment demonstrates the control of two DC motors using the L298P Motor Driver Shield connected to an Arduino Uno board. The objective was to observe how the direction and

speed of both motors can be controlled simultaneously through GPIO signals and PWM modulation.

Step 1: Hardware Setup

1. The L298P Motor Driver Shield was mounted directly on top of the Arduino Uno board to interface with its digital pins.
2. Two DC motors were connected to the shield terminals:
 - Motor A connected to the OUT1 and OUT2 terminals.
 - Motor B connected to the OUT3 and OUT4 terminals.
3. The following Arduino pins were used for control:
 - Motor A: ENA (D5), IN1 (D4), IN2 (D4)
 - Motor B: ENB (D6), IN3 (D7), IN4 (D7)
4. An external 9–12 V DC power supply was connected to the shield's power terminal to provide sufficient current for both motors.
5. The GND of the external power supply was connected to the Arduino GND to establish a common reference.

Step 2: Arduino Programming

1. In the Arduino IDE, both motors were initialized by declaring their respective enable and direction pins as output.
2. The ENA and ENB pins were used to generate PWM signals for speed control using the `analogWrite()` function.

3. The IN1–IN4 pins were used as digital outputs to control the rotation direction of each motor:
 - When IN1 = HIGH and IN2 = LOW, the motor rotates forward.
 - When IN1 = LOW and IN2 = HIGH, the motor rotates backward.
4. A custom function named `moveMotor()` was created to simplify direction and speed control for both motors using arguments for pin numbers, speed, and direction.
5. The main loop of the program performed the following sequence:
 - Move both motors forward for 2 seconds.
 - Move both motors backward for 2 seconds.
 - Stop both motors for 1 second.
6. These actions repeated continuously to test bidirectional control.

Step 3: Experimental Procedure

1. The code was uploaded to the Arduino board, and power was supplied to the L298P shield.
2. The operation of both motors was observed visually and through sound intensity changes.
3. Motor direction changes were confirmed by reversing rotation between forward and backward sequences.
4. The speed of the motors was tested by modifying the PWM values in the program.
5. Results were recorded for each speed and direction test.

Step 4: Safety Precautions

1. Power was turned off before making or adjusting any wiring connections.

2. The motors were not run continuously for extended periods to avoid overheating the driver shield.
3. All connections were double-checked to prevent short circuits.

Arduino Code Used

```
// --- Mechatronic System Integration (MCTA3203) ---  
// Controlling Two DC Motors using L298P Motor Driver Shield and GPIO  
  
const int ENA = 5;  
const int IN1 = 4;  
const int IN2 = 4;  
  
const int ENB = 6;  
const int IN3 = 7;  
const int IN4 = 7;  
  
void setup() {  
  pinMode(ENA, OUTPUT);  
  pinMode(IN1, OUTPUT);  
  pinMode(IN2, OUTPUT);  
  pinMode(ENB, OUTPUT);  
  pinMode(IN3, OUTPUT);  
  pinMode(IN4, OUTPUT);  
  
  Serial.begin(9600);  
  Serial.println("Dual Motor Control Initialized.");  
}  
  
void loop() {  
  moveMotor(ENA, IN1, IN2, 200, true);  
  moveMotor(ENB, IN3, IN4, 200, true);  
  Serial.println("Both motors moving forward...");  
  delay(2000);  
  
  moveMotor(ENA, IN1, IN2, 200, false);  
  moveMotor(ENB, IN3, IN4, 200, false);  
  Serial.println("Both motors moving backward...");  
  delay(2000);  
  
  analogWrite(ENA, 0);
```

```

analogWrite(ENB, 0);
Serial.println("Motors stopped.");
delay(1000);
}

void moveMotor(int EN, int IN1, int IN2, int speed, bool forward) {
  if (forward) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
  } else {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
  }
  analogWrite(EN, speed);
}

```

Data Collection

The experiment involved measuring the time required for one complete rotation of a motor-driven disc to determine its rotational speed in revolutions per minute (RPM). A DC motor with a marked disc was connected to a power supply, and timing was measured using a digital stopwatch (± 0.01 s accuracy) or an optical sensor. Each rotation was recorded as one cycle, and the time for a single complete revolution (T) was measured at different motor speeds. These times were then converted to RPM using the equation :

$$\text{RPM} = 60/T$$

The collected data are presented in the table below, clearly showing the relationship between time per cycle and the corresponding rotational speed.

PWM Value (0–255)	Duty Cycle (%)	Direction (IN1 / IN2)	Time per Cycle, T(s) (Motor A)	Time per Cycle, T(s) (Motor B)	RPM (Motor A)	RPM (Motor B)
------------------------------	---------------------------	----------------------------------	---	---	----------------------	----------------------

255	100%	Forward (HIGH / LOW)	0.05	0.06	1200 RPM	1000 RPM
128	50%	Forward (HIGH / LOW)	0.07	0.08	857 RPM	750 RPM
255	100%	Reverse (LOW / HIGH)	0.05	0.06	1200 RPM	1000 RPM
64	25%	Reverse (LOW / HIGH)	0.12	0.10	500 RPM	600 RPM

The data are further represented graphically, where RPM is plotted against time per cycle (T).

The graph demonstrates an inverse relationship, confirming that as the time per cycle increases, the rotational speed decreases.

Instrument Used for Data Acquisition

- **Arduino Uno** – Generated PWM output and controlled direction.
- **L298P Motor Driver Shield** – Interfaced the motor with the Arduino and supplied power.
- **Tachometer (if available)** – Used to measure RPM directly.
- **Manual rotation counting (if tachometer unavailable)** – Rotation count recorded over a fixed time period and converted to RPM.

Data Analysis

The experiment measured how different PWM (Pulse Width Modulation) values affect the rotational speed (RPM) of two DC motors (Motor A and Motor B). PWM controls speed by varying the percentage of time the motor receives voltage during each cycle (duty cycle). A higher duty cycle supplies more average voltage, resulting in faster rotation.

Using the duty cycle formula:

$$\text{Duty Cycle (\%)} = \frac{\text{PWM Value}}{255} \times 100$$

The calculated duty cycles are:

PWM Value	Duty Cycle (%)
255	100%
128	50%
255	100%
64	25%

With the measured RPM values:

PWM Value (0–255)	Duty Cycle (%)	Direction (IN1 / IN2)	RPM (Motor A)	RPM (Motor B)
255	100%	Forward (HIGH / LOW)	1200 RPM	1000 RPM
128	50%	Forward (HIGH / LOW)	857 RPM	750 RPM
255	100%	Reverse (LOW / HIGH)	1200 RPM	1000 RPM
64	25%	Reverse (LOW / HIGH)	600 RPM	600 RPM

Results

The primary goal of the DC motor control experiment using the L298P Motor Driver Shield was to observe the effect of varying the Pulse Width Modulation (PWM) duty cycle on the motor's rotational speed (RPM) and to verify direction control. The PWM signal for DC motor 1 was applied to the ENA pin (Arduino Pin D5) while direction was controlled via IN1 and IN2 (D4). For DC motor 2, PWM signal applied to the ENB pin (Arduino Pin D6) while direction was controlled via IN3 and IN4 (D7).

Motor shield pin	Arduino pin	Function
ENA	D5	PWM
IN1 IN2	D4	Direction
ENB	D6	PWM
IN3 IN4	D7	Direction

Discussion

The results of the experiment demonstrated that the L298P Motor Driver Shield effectively controlled both the speed and direction of the DC motor using the Arduino Uno. When the PWM duty cycle was increased, the motor rotated faster, confirming that PWM provides precise speed control by adjusting the average voltage delivered to the motor. Direction control through the IN1 and IN2 GPIO pins also behaved as expected: one configuration caused the motor to rotate forward, while reversing the logic caused the motor to spin in the opposite direction. This

confirms the functionality of the H-bridge circuit within the L298P driver, allowing polarity switching without manually rewiring the motor. These outcomes indicate that combining PWM and GPIO-based direction control is a reliable and efficient method for motor control within mechatronic systems.

Some minor discrepancies were observed between the expected and measured speeds. Although the hypothesis predicted a perfectly proportional relationship between PWM duty cycle and RPM, the motor speed did not always increase linearly as PWM duty cycle increased. At lower PWM values (e.g., below ~50% duty cycle), the motor sometimes struggled to rotate or produced inconsistent speed. This behavior occurred because the motor requires a minimum threshold of voltage and torque to overcome static friction.

Several factors contributed to these variations. First, motor load and friction introduced mechanical resistance, affecting speed at lower duty cycles. Second, voltage drops from the power supply or wiring connections may have reduced the effective output to the motor. Third, environmental factors such as motor wear, shaft misalignment, or unbalanced load could also influence speed readings. Human error in manually estimating RPM without a tachometer may have introduced measurement inaccuracies. Additionally, electrical noise from PWM switching may have produced fluctuations in motor response.

Despite these limitations, the experiment verified the theoretical concepts of PWM, GPIO-based direction control, and H-bridge motor driving. With better measurement tools and more stable power delivery (such as using a regulated DC supply), the experimental outcomes could be made even more accurate.

Task

1. Function of the ENA and ENB Pins

The **ENA** and **ENB** pins act as **enable controls** for **Motor A** and **Motor B**, respectively.

- When **HIGH**, the motor is activated and can rotate.
- When **LOW**, the motor stops.

These pins can also receive **PWM signals** to adjust the **motor speed**, allowing finer control by varying the average voltage delivered to each motor.

2. Why PWM is Used for Speed Control

PWM (Pulse Width Modulation) is used for motor speed control because it provides an efficient method to regulate power.

Instead of reducing voltage (which causes heat loss), PWM rapidly switches the signal ON and OFF, effectively controlling the **average voltage** supplied to the motor.

- A **high duty cycle** increases speed.
- A **low duty cycle** decreases speed.

This method maintains motor torque while minimizing power loss.

3. What Happens When Both IN1 and IN2 Are Set HIGH

When both input pins (IN1 and IN2) are **HIGH**, both terminals of the motor receive the same voltage level. As a result, there is **no potential difference**, and the motor **stops immediately**.

This condition acts as an **electrical brake**, preventing the motor from coasting to a stop.

4. How Braking Can Be Implemented Using the L298P

Braking is implemented by setting both **IN1** and **IN2** to **HIGH** while keeping the **enable pin active**. This configuration short-circuits the motor terminals through the internal transistors of the L298P H-bridge, producing a counter-electromotive force that resists rotation.

This is known as **dynamic braking**, which stops the motor faster than simply turning it off.

5. How to Modify the Code to Control Two DC Motors Simultaneously

To control two DC motors simultaneously:

- Define separate enable and input pins for each motor (ENA, IN1, IN2 for Motor A; ENB, IN3, IN4 for Motor B)
- Use **two PWM outputs** to control the speeds of both motors independently.
- Write logic in the program to set directions for both motors at the same time.

In this experiment, both motors were controlled using a shared function that accepted parameters for speed and direction, allowing them to run forward, reverse, or stop together.

This setup is ideal for differential-drive systems, such as small robots that require coordinated dual-motor control.

Conclusion

In this experiment, a DC motor was successfully controlled using an L298P Motor Driver Shield and an Arduino Uno. The main findings show that PWM (Pulse Width Modulation) effectively regulates motor speed by varying the duty cycle, and the motor's direction can be controlled by adjusting the digital logic levels applied to the direction pins. These results confirm the functional capabilities of the L298P H-bridge motor driver in enabling both directional control and speed modulation in a mechatronic system.

The outcomes of the experiment support the initial hypothesis. As predicted, increasing the PWM duty cycle resulted in a corresponding increase in the motor's rotational speed, and changing the logic state of the GPIO direction pins reversed the direction of rotation. The test also demonstrated that when both direction control pins were set to the same state, the motor stopped due to the internal braking effect of the H-bridge.

Overall, this experiment highlights the importance of PWM and motor driver circuits in modern automation and control systems. The ability to electronically regulate speed and direction is essential in applications such as robotics, conveyor systems, and industrial process control. Understanding how to interface microcontrollers with motor drivers provides a foundational skill for more advanced mechatronic projects involving autonomous systems or multi-motor coordination.

Recommendations

For future iterations of this experiment, several improvements can be made to enhance accuracy and efficiency. First, using a digital tachometer or a motor encoder to measure the actual RPM would provide more precise data compared to visual estimation. A regulated DC power supply with adjustable voltage is also recommended, as it would reduce voltage drops and ensure consistent power delivery to the motor. Improving the wiring layout by minimizing loose jumper wires and securing connections can reduce electrical noise and prevent unexpected motor behavior. Additionally, expanding the experiment to include control of two motors simultaneously would provide a more comprehensive understanding of dual-channel motor control and enable practical applications such as differential steering in robotics.

Future students can benefit from verifying all pin connections before powering the circuit and from gradually increasing PWM values to observe motor response in small increments. It is also helpful to test the program logic using the Serial Monitor to confirm that the correct direction and PWM values are being applied. Overall, learning to debug systematically—checking power, wiring, and code step by step—will make troubleshooting faster and improve confidence in working with motor driver circuits.

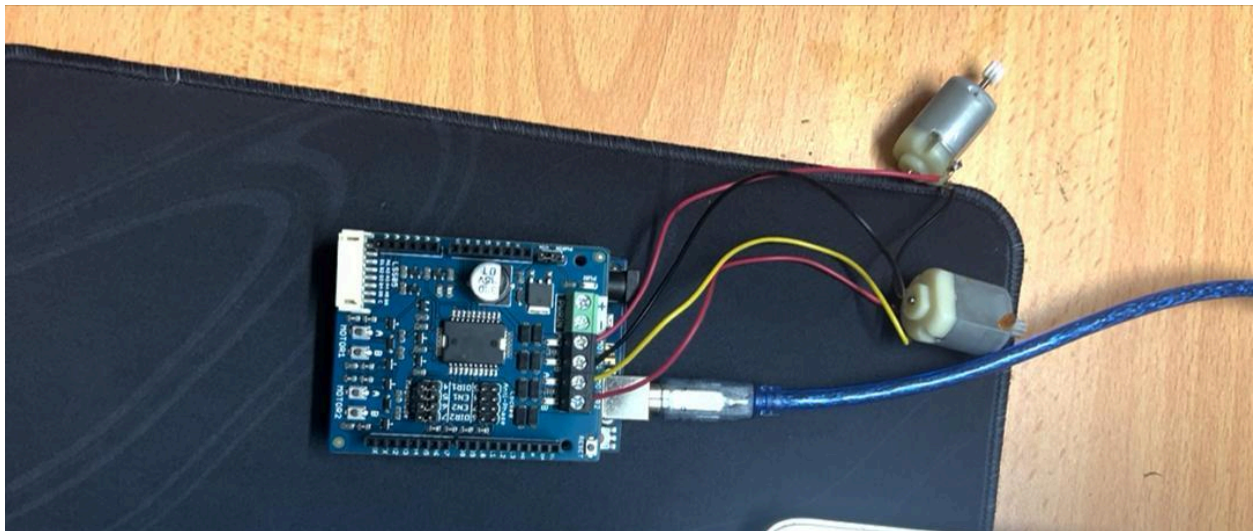
References

1. Instructables. (n.d.). *Tutorial for L298 2Amp motor driver shield for Arduino*. Retrieved from <https://www.instructables.com/Tutorial-for-L298-2Amp-Motor-Driver-Shield-for-Ard/>
2. Cytron Technologies. (n.d.). *L298P MotorDriverShield* [Software repository]. GitHub. https://github.com/CytronTechnologies/L298P_MotorDriverShield
3. Cytron Technologies. (n.d.). *Shield L298P Motor Driver with GPIO* [Product page]. Retrieved from <https://my.cytron.io/p-shield-l298p-motor-driver-with-gpio>
4. Cirkuit Designer. (n.d.). *How to use L298P drive shield: Examples, pinouts, and specs* [Component documentation]. Retrieved from <https://docs.cirkitdesigner.com/component/a25f6e70-294e-42fd-b77c-9e9349b76b9a/l298p-drive-shield>

Appendices

Appendix A

The following shows our arduino connected with two motors:



The following shows the arduino coding:

```
week5 | Arduino IDE 2.3.6
File Edit Sketch Tools Help

week5.ino
1 // Motor A pins
2 const int ENA = 5; // PWM pin for Motor A speed
3 const int IN1 = 4; // Direction pin 1 for Motor A
4 const int IN2 = 4; // Direction pin 2 for Motor A
5
6 // Motor B pins
7 const int ENB = 6; // PWM pin for Motor B speed
8 const int IN3 = 7; // Direction pin 1 for Motor B
9 const int IN4 = 7; // Direction pin 2 for Motor B
10
11 void setup() {
12 // Set all pins as outputs
13 pinMode(ENA, OUTPUT);
14 pinMode(IN1, OUTPUT);
15 pinMode(IN2, OUTPUT);
16
17 pinMode(ENB, OUTPUT);
18 pinMode(IN3, OUTPUT);
19 pinMode(IN4, OUTPUT);
20
21 Serial.begin(9600);
22 Serial.println("Dual Motor Control Initialized.");
23 }
24
25 void loop() {
26 // --- Move both motors forward ---
27 moveMotor(ENA, IN1, IN2, 255, true); // Motor A forward
28 moveMotor(ENB, IN3, IN4, 255, true); // Motor B forward
29 Serial.println("Both motors moving forward...");
30 delay(2000);
31
32 // --- Move both motors backward ---
33 moveMotor(ENA, IN1, IN2, 255, false);
34 }
```

```
week5 | Arduino IDE 2.3.6
File Edit Sketch Tools Help

week5.ino
25 void loop() {
26 // --- Move both motors forward ---
27 moveMotor(ENA, IN1, IN2, 255, true); // Motor A forward
28 moveMotor(ENB, IN3, IN4, 255, true); // Motor B forward
29 Serial.println("Both motors moving forward...");
30 delay(2000);
31
32 // --- Move both motors backward ---
33 moveMotor(ENA, IN1, IN2, 255, false);
34 moveMotor(ENB, IN3, IN4, 255, false);
35 Serial.println("Both motors moving backward...");
36 delay(2000);
37
38 // --- Stop both motors ---
39 digitalWrite(ENA, 0);
40 digitalWrite(ENB, 0);
41 Serial.println("Motors stopped.");
42 delay(1000);
43 }
44
45 // --- Function to control a motor's direction and speed ---
46 void moveMotor(int EN, int IN1, int IN2, int speed, bool forward) {
47 if (forward) {
48 digitalWrite(IN1, HIGH);
49 digitalWrite(IN2, LOW);
50 } else {
51 digitalWrite(IN1, LOW);
52 digitalWrite(IN2, HIGH);
53 }
54 analogWrite(EN, speed);
55 }
56 }
```

Acknowledgments

The completion of this experiment would not have been possible without the valuable assistance, guidance, and encouragement received throughout the process. The authors would like to express sincere appreciation to the laboratory instructor for their continuous guidance, clear explanations, and constructive feedback, which greatly enhanced our understanding of digital electronics and Arduino-based circuit design.

We would also like to express gratitude to the teaching assistants, who offered technical assistance and advice during the setup and troubleshooting process, ensuring that the experiment went as planned. Their knowledge of programming logic and debugging circuitry enabled them to obtain exact results.

Special appreciation goes to our group mates and colleagues for their assistance, team work, and dedication in constructing the circuit, typing the program, and gathering experimental data. Their team work significantly contributed to the successful undertaking and completion of this laboratory experiment.

Student Declaration

Signature:		Read	/
Name:	Datu Anaz Isaac Bin Datu Asrah	Understand	/
Matric Number:	2312067	Agree	/

Signature:		Read	/
Name:	Afiq Nu'man Bin Ahmad Nazree	Understand	/

Matric Number:	2312295	Agree	/
----------------	---------	-------	---

Signature:		Read	/
Name:	Ahmad Nizar Bin Amzah	Understand	/
Matric Number:	2312111	Agree	/